RAG-BASED CUSTOMER SUPPORT ASSISTANT

with LangGraph & Human-in-the-Loop (HITL)

DELIVERABLE 1

HIGH-LEVEL DESIGN (HLD)

Internship Project Document

Domain: AI / Machine Learning

Technology Stack: Python · FastAPI · LangGraph · ChromaDB · Groq LLM

1. System Overview

1.1 Problem Definition

Modern businesses struggle to provide instant, accurate, and scalable customer support. Traditional systems either rely on rigid keyword-based FAQs that fail to handle natural language, or require human agents for every query which is expensive and slow. Large Language Models (LLMs) alone also suffer from hallucination — generating plausible but incorrect answers when asked about specific company policies, products, or procedures.

This project addresses the problem by designing and implementing a Retrieval-Augmented Generation (RAG) system that grounds the LLM's responses in a verified company knowledge base (PDF documents). The result is a system that is simultaneously accurate, contextual, scalable, and safe — routing complex or sensitive queries to human agents via a Human-in-the-Loop (HITL) mechanism.

1.2 Scope of the System

The system is scoped to deliver the following capabilities:

- Accept PDF documents as the primary knowledge base input
- Process and store document embeddings in a persistent local vector database (ChromaDB)
- Accept natural language queries from users via a web-based chat interface
- Retrieve semantically relevant document chunks using cosine similarity search
- Generate accurate, grounded answers using an LLM (Claude via Anthropic API)
- Route queries intelligently based on detected intent (greeting, knowledge, escalation)
- Escalate sensitive or low-confidence queries to human agents with full ticket tracking
- Provide a human agent dashboard within the same frontend for HITL ticket management

2. Architecture Diagram (Mandatory)

2.1 Overall System Architecture

The system is built across four horizontal layers: the Browser Frontend, the FastAPI Backend, the Intelligence/RAG Pipeline, and the Persistence Layer. Each layer communicates with its adjacent layer through well-defined interfaces.

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                LAYER 1: USER INTERFACE                                                 │
│                          HTML / CSS / JavaScript (Browser)                                            │
│                                                                                                       │
│  ┌──────────────────────────────┐   ┌──────────────────────────────────────────────────────────┐    │
│  │  Chat Window                 │   │  HITL Agent Dashboard                                    │    │
│  │  User sends query            │   │  Agent sees & resolves tickets                           │    │
│  └──────────────────────────────┘   └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
          │ POST /chat                          │ GET /hitl/tickets
          │ POST /ingest                        │ POST /hitl/respond
          ▼                                     ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                LAYER 2: FastAPI BACKEND                                               │
│  Endpoints: /chat  /ingest  /status  /hitl/tickets  /hitl/respond                                     │
│                                                                                                       │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────────┐    │
│  │                          LAYER 3: LangGraph Workflow Engine                                  │    │
│  │                                                                                              │    │
│  │  [intent_router] ──► [retrieve] ──► [grade_docs] ──► [generate]                              │    │
│  │                                                                                              │    │
│  │                                                      ──► [escalate_hitl]                     │    │
│  │                                                                                              │    │
│  │  ──► [simple_reply]  (for greetings & farewells)                                             │    │
│  └──────────────────────────────────────────────────────────────────────────────────────────────┘    │
│                                                                                                       │
│  ┌──────────────────────────────┐  ┌──────────────────────────────┐  ┌──────────────────────────────┐  │
│  │ DocumentProcessor            │  │ EmbeddingModel               │  │ LLMHandler                   │  │
│  │ PyPDF + Splitter             │  │ all-MiniLM-L6                │  │ Claude (Anthropic)           │  │
│  └──────────────────────────────┘  └──────────────────────────────┘  └──────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
                  │ embed() / query()
                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                LAYER 4: PERSISTENCE LAYER                                             │
│                                                                                                       │
│  ┌──────────────────────────────────────────────┐  ┌──────────────────────────────────────────┐    │
│  ChromaDB (Local Vector Store)                  │  │ HITL JSON Store                          │    │
│  - Cosine similarity search                     │  │ - Ticket creation                        │    │
│  - Persistent embeddings                        │  │ - Agent responses                        │    │
│  - Metadata storage                             │  │ - Status tracking                        │    │
│  └──────────────────────────────────────────────┘  └──────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

2.2 Document Ingestion Pipeline

The ingestion pipeline runs once when a PDF is uploaded. It transforms raw PDF text into searchable vector embeddings:

```
PDF File
  │
  ▼ PyPDFLoader
List of Pages (one Document per page)
  │
  ▼ RecursiveCharacterTextSplitter (chunk_size=500, overlap=50)
List of Chunks (each with page_content + metadata)
  │
  ▼ SentenceTransformer (all-MiniLM-L6-v2)
List of Embedding Vectors (384-dimensional floats)
  │
  ▼ ChromaDB.upsert()
Persisted in chroma_db/ on disk
```

3. Component Descriptions

3.1 Document Loader

The Document Loader is implemented using LangChain's PyPDFLoader, which wraps the pypdf library. It reads each page of the uploaded PDF and returns a list of LangChain Document objects, one per page. Each document carries the raw text content and metadata including the source filename and page number. The loader handles multi-page documents of any length without requiring manual page selection.

3.2 Chunking Strategy

The chunking module uses RecursiveCharacterTextSplitter with a chunk size of 500 characters and an overlap of 50 characters. This splitter attempts to break text at natural semantic boundaries — first trying paragraph breaks (double newlines), then single newlines, then sentence endings, then word spaces, and finally falling back to raw character-level splitting. This hierarchy preserves the semantic coherence of each chunk better than simple fixed-size splitting.

The 50-character overlap ensures that if a key sentence falls at the boundary between two chunks, it appears in both — preventing information loss at chunk edges. Each chunk is enriched with metadata including its sequential chunk index, source filename, and originating page number.

3.3 Embedding Model

The embedding model used is all-MiniLM-L6-v2, accessed locally via the sentence-transformers library. It produces 384-dimensional dense vector representations of text, optimized for semantic similarity tasks. The model runs entirely on CPU, requires no API key or network call, and processes text in batches of 32 for efficiency. It was pre-trained on over 1 billion sentence pairs, making it highly suitable for question-answer retrieval scenarios.

3.4 Vector Store

ChromaDB is used as the persistent vector database. It runs locally and persists embeddings to the chroma_db/ directory on disk. The collection is configured with cosine distance as the similarity metric, which is ideal for text embeddings because it measures the angle between vectors (semantic direction) rather than their magnitude. ChromaDB supports upsert operations, which means re-ingesting the same PDF will update existing chunks rather than creating duplicates.

3.5 Retriever

The Retriever module ties the embedding model and vector store together. Given a user query, it first embeds the query using the same EmbeddingModel used during ingestion. It then queries ChromaDB for the top-K most similar chunks by cosine distance. Chunks with a distance above the configured threshold (0.35 by default) are filtered out as irrelevant. A confidence score is computed as 1 minus the mean distance of the remaining results, providing a 0-to-1 quality signal for downstream routing decisions.

3.6 Graph Workflow Engine

LangGraph's StateGraph is used to orchestrate the entire query-response pipeline. The graph consists of six nodes connected by directed edges with conditional branching. The state is a typed Python TypedDict that flows through all nodes, accumulating fields as each node executes. The compiled graph is a callable object that accepts an initial state and returns the final state after traversal. This architecture makes the control flow explicit, testable, and easy to extend.

3.7 Routing Layer

The Router classifies user queries into one of four intents: GREETING, FAREWELL, KNOWLEDGE, or ESCALATION. It applies a deterministic rule-based approach in priority order: first checking for greeting/farewell keywords, then scanning for escalation trigger words (refund, legal, manager, fraud, etc.), and finally evaluating retrieval confidence. This layer is called twice in the graph — once before retrieval (fast pre-classification) and once after retrieval (to incorporate real confidence data).

3.8 HITL Module

The Human-in-the-Loop module manages the lifecycle of escalation tickets. When triggered, it generates a unique 8-character ticket ID, records the query, reason, session ID, and timestamp, and persists the ticket to a JSON file. The module exposes methods to resolve tickets (agent submits response), retrieve single tickets, and list all pending tickets. The frontend agent dashboard polls these endpoints to display and manage tickets in real time.

4. Data Flow

4.1 How Data Moves from PDF to Answer

The data flow occurs in two distinct phases: the Ingestion Phase (one-time setup) and the Query Phase (per user message).

Phase A — PDF Ingestion:

1. User uploads a PDF file via the web interface (POST /ingest)
2. FastAPI saves the file to a temporary location on disk
3. DocumentProcessor loads the PDF with PyPDFLoader, extracting text per page
4. RecursiveCharacterTextSplitter divides pages into overlapping 500-character chunks
5. EmbeddingModel encodes all chunks into 384-dimensional vectors (batch processing)
6. VectorStore.add_documents() upserts all chunks and embeddings into ChromaDB
7. ChromaDB persists the data to chroma_db/ directory
8. API returns ingestion statistics (chunk count, size stats) to the frontend

Phase B — Query Lifecycle:

9. User types a question in the chat window (POST /chat)
10. FastAPI receives ChatRequest with query and session_id
11. RAGWorkflow.run() is called, initializing an empty GraphState
12. Node: intent_router — detects intent (greeting/farewell/escalation/knowledge)
13. If GREETING or FAREWELL: simple_reply node returns a canned message → END
14. If ESCALATION keyword detected: escalate_hitl node creates ticket → END
15. If KNOWLEDGE: retrieve node embeds query → queries ChromaDB → gets top-5 chunks
16. Node: grade_documents — filters chunks by threshold, computes confidence score
17. If confidence < 0.50: escalate_hitl node creates ticket → END
18. Node: generate_answer — calls Claude with context blocks and user query
19. Groq returns a grounded, factual answer string
20. Final GraphState with answer, intent, confidence, and escalation info is returned
21. FastAPI serializes GraphState to ChatResponse JSON and sends to browser
22. JavaScript renders the message bubble with metadata badges in the chat window

5. Technology Choices

5.1 Why ChromaDB

Factor	Details
Cost	Completely free and open-source — zero licensing or API cost
Deployment	Runs locally, no cloud infrastructure or network latency
Persistence	PersistentClient stores data to disk between restarts
Performance	HNSW index enables fast approximate nearest neighbour search
Similarity	Native cosine distance support — ideal for text embeddings
API	Simple Python API for upsert, query, delete, and count
Alternatives considered	FAISS (no persistence), Pinecone (cloud cost), Milvus (complex setup)

5.2 Why LangGraph

Factor	Details
Graph model	Represents the pipeline as a directed state graph — readable and maintainable
Typed state	TypedDict state ensures all nodes share a well-defined contract
Conditional edges	Native support for branching logic (escalate vs. generate)
Extensibility	New nodes (e.g., memory, re-ranker) can be added without refactoring
Debugging	Graph structure makes it easy to trace which node handled a query
Alternative considered	Simple if-else chain — less readable, harder to test and extend

5.3 Additional Tools

Tool	Purpose
FastAPI	High-performance async Python web framework with auto-generated docs
Sentence-Transformers	Local, CPU-friendly embedding model (all-MiniLM-L6-v2)
LangChain	Document loaders and text splitters for PDF processing
pypdf	PDF text extraction backend for PyPDFLoader
Pydantic	Request/response validation and serialization in FastAPI
Python-dotenv	Environment variable management for API keys
Vanilla JS + CSS	Zero-dependency frontend — no build step, easy to deploy

6. Scalability Considerations

6.1 Handling Large Documents

In the current implementation, PDFs are processed synchronously on upload. For large documents (100+ pages), this creates a blocking delay. The following design decisions address this at different scales:

- Chunk size of 500 characters keeps embedding batches manageable, preventing memory overflow on large files
- ChromaDB's upsert operation means the same document can be re-ingested without creating duplicates
- For production at scale, ingestion should be moved to an async background task queue (e.g., Celery with Redis) so the HTTP response returns immediately with a job ID, and status is polled
- Multi-document support can be achieved by using separate ChromaDB collections per document or per customer, namespaced by a tenant ID

6.2 Handling Increasing Query Load

Load Level	Current Design	Production Upgrade
1-10 RPS	FastAPI handles fine (sync workers)	No change needed
10-100 RPS	Bottleneck at LLM API call (~500ms per call)	Response caching for common queries (Redis)
100+ RPS	ChromaDB single-node limit	Migrate to Pinecone or Weaviate with horizontal scaling
1000+ RPS	Single-server FastAPI bottleneck	Load balancer + multiple FastAPI instances + async LLM calls

6.3 Latency Concerns

The system has three primary latency contributors, ordered by typical impact:

23. LLM API call: 300-800ms — dominates total response time. Mitigation: use streaming (streamed tokens appear progressively), cache common answers in Redis with a TTL
24. Embedding: 20-80ms for a single query on CPU. Mitigation: use GPU for the embedding model in production; batch embeddings during ingestion
25. ChromaDB query: 5-20ms for up to 50K vectors — generally negligible. At 1M+ vectors, migrate to a distributed vector database

Target response time for the complete pipeline (embed + retrieve + generate) is under 1.5 seconds for typical queries.